

Apache Paimon Flink-1.20 模块

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-flink-1.20 (release-1.3.1)

生成工具：AI Architecture Analyzer

生成日期：2026年03月26日

第章 目录

第1章 模块概述 · · ·	模块定位 / 核心目标 / 三大依赖模块
第2章 核心架构设计 · · ·	四层分层架构 / 模块依赖 / SPI扩展点
第3章 核心子系统详解 · · ·	Catalog / Source / Sink / Action / Procedure / Lookup / CDC
第4章 设计模式详解 · · ·	Factory / Template / Strategy / Builder / SPI / 2PC / Adapter
第5章 核心流程图 · · ·	读取流程 / 写入流程 / CDC同步流程
第6章 核心时序图 · · ·	Source创建 / Sink提交 / CDC整库同步
第7章 关键场景调用链 · · ·	表创建 / 数据读取 / 数据写入 / CDC同步 / Compaction
第8章 设计亮点与总结 · · ·	架构评估 / 质量评分

第1章 模块概述

1.1 模块定位

paimon-flink-1.20 是 Apache Paimon 面向 Flink 1.20.x 的适配集成模块。该模块本身是纯打包(Shading)模块，不含源代码，通过 maven-shade-plugin 将 paimon-flink-common、paimon-flink1-common、paimon-flink-cdc 三个核心依赖打包为一个可部署的 Fat JAR。用户只需引入此 JAR 即可获得完整的 Paimon + Flink 集成能力。

1.2 核心目标

- Flink SQL/Table API 集成：通过 FlinkCatalog + FlinkTableFactory 实现完整 DDL/DML 支持
- DataStream API 集成：通过 FlinkSourceBuilder/FlinkSinkBuilder 提供编程式读写
- 批流一体读写：Batch(StaticFileStoreSource) + Streaming(ContinuousFileStoreSource)
- CDC 实时同步：支持 MySQL/PostgreSQL/MongoDB/Kafka/Pulsar 五种数据源
- Action CLI：30+ 命令行操作（Compact/MergeInto/Delete/Clone/Migrate等）
- Procedure 框架：40+ SQL Procedure（compact/create_tag/repair等）
- Lookup Join：维表关联查询，支持全缓存/主键/二级索引等策略

1.3 三大依赖模块概览

模块	源文件数	核心职责
paimon-flink-common	460+	Flink集成核心：Catalog/Source/Sink/Action/Procedure/Lookup/类型转换
paimon-flink1-common	16	Flink 1.x兼容层：Variant类型Stub/RowData增强/RuntimeContext/参数适配
paimon-flink-cdc	170+	CDC同步：MySQL/PG/MongoDB/Kafka/Pulsar + Schema演进 + Pipeline集成

1.4 版本适配机制

paimon-flink-1.20 通过 flink.version=1.20.1 声明目标版本，paimon-flink1-common 提供 Flink 1.x/2.x 兼容层，包括 RowData/ArrayData 接口增强（getVariant）、RuntimeContextUtils 和 MultipleParameterToolAdapter。

第2章 核心架构设计

2.1 整体四层架构

paimon-flink 集成层采用四层架构：SQL/Table API 层、DataStream API 层、算子/运行时层、Paimon Core 存储引擎，层间通过接口解耦。

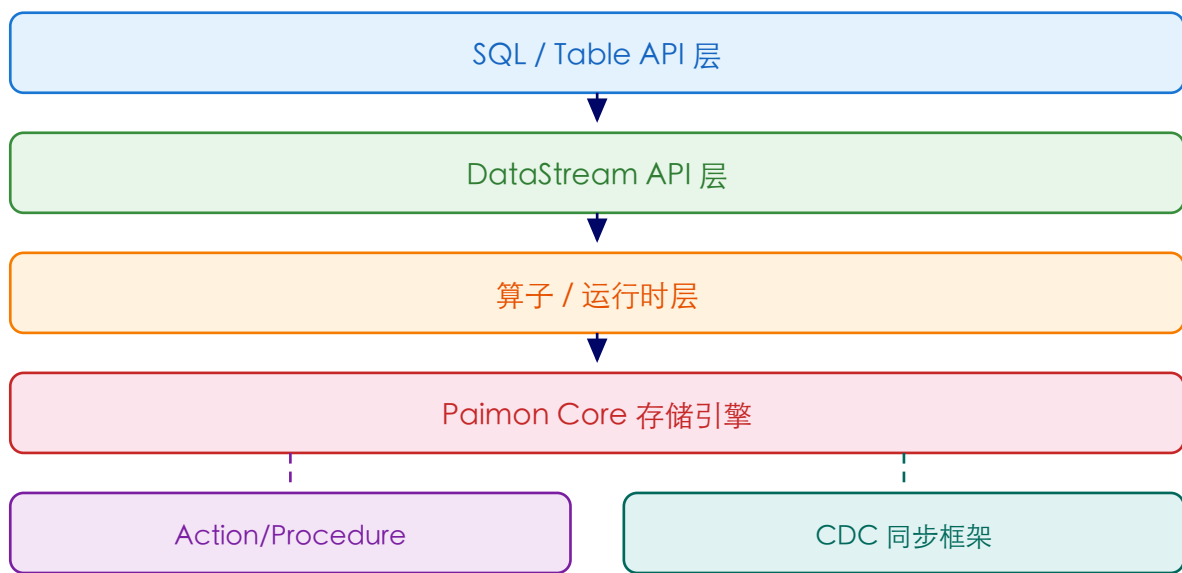


图2-1 整体四层架构图

2.2 模块依赖关系

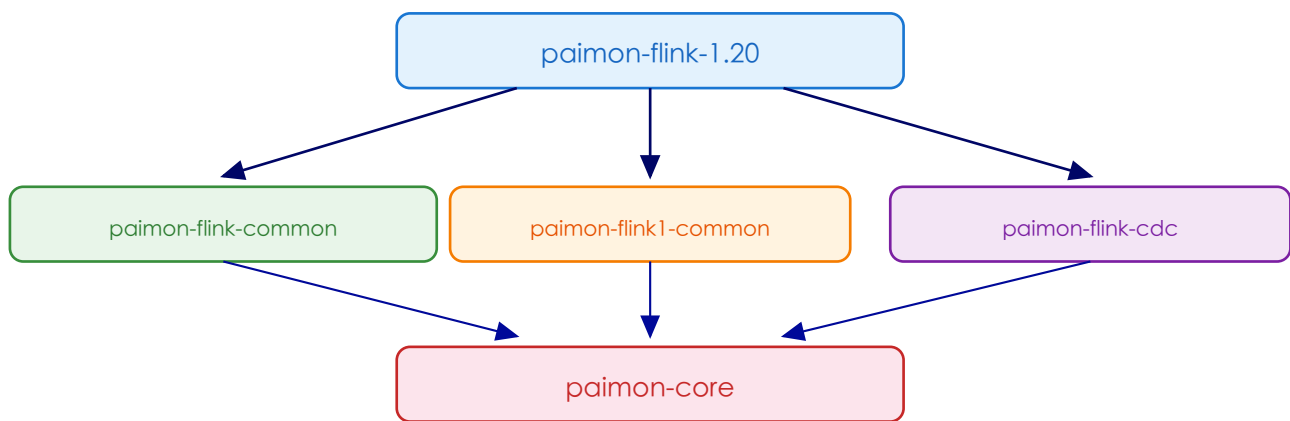


图2-2 模块依赖关系图

2.3 SPI 扩展点注册

SPI 接口	identifier	注册实现	说明
CatalogFactory	paimon	FlinkCatalogFactory	创建FlinkCatalog

SPI 接口	identifier	注册实现	说明
DynamicTableSourceFactory	paimon	FlinkTableFactory	创建Source
DynamicTableSinkFactory	paimon	FlinkTableFactory	创建Sink
ActionFactory	多个	CompactActionFactory等30+	CLI Action
ProcedureBase	多个	CompactProcedure等40+	SQL Procedure

第3章 核心子系统详解

3.1 Catalog 子系统

FlinkCatalog 继承 Flink AbstractCatalog，包装 Paimon 原生 Catalog。FlinkCatalogFactory 通过 SPI (identifier="paimon") 在 CREATE CATALOG 时自动创建实例。

- Database管理：createDatabase / dropDatabase / listDatabases / getDatabase
- Table管理：createTable / dropTable / alterTable / listTables / getTable
- Partition管理：listPartitions / getPartition / dropPartition
- Function管理：listFunctions 返回系统函数和已注册 Procedure 列表
- 类型转换：FlinkRowData (Paimon->Flink) 、FlinkRowWrapper (Flink->Paimon)

FlinkTableFactory 统一实现 DynamicTableSourceFactory/DynamicTableSinkFactory，还支持 auto-create——INSERT 时表不存在则自动创建。

3.2 Source 子系统

Source 采用三层继承：FlinkTableSource -> BaseDataTableCellSource -> DataTableCellSource，逐层叠加 PushDown 能力。

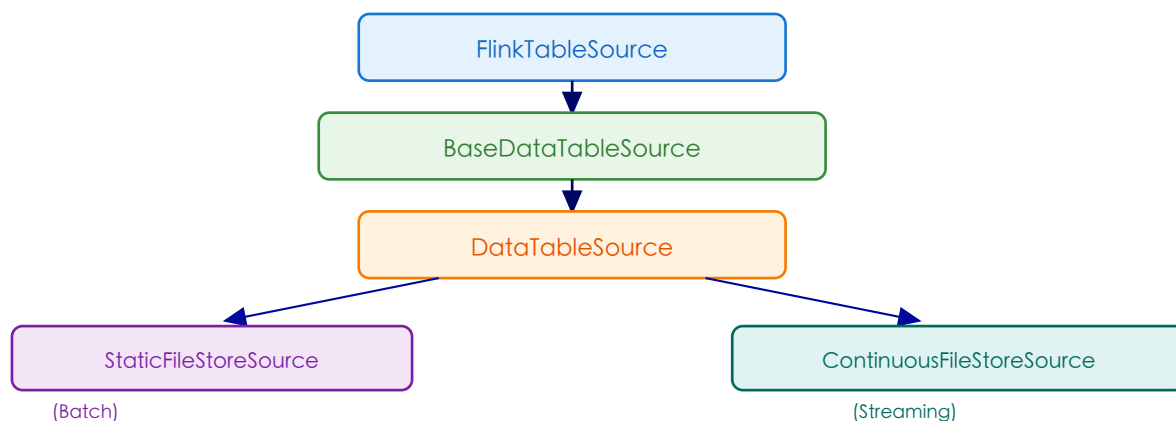


图3-1 Source 类继承体系

- FlinkTableSource：Filter/Projection/Limit PushDown
- BaseDataTableCellSource：+Watermark/Lookup/Aggregate PushDown
- DataTableCellSource：+Statistics 报告 + 动态分区过滤
- StaticFileStoreSource：有界Batch读取 ContinuousFileStoreSource：无界Streaming读取

3.3 Sink 子系统

Sink 基于 Flink 两阶段提交（2PC）保证 Exactly-Once。Table API 路径：FlinkTableSinkBase -> SupportsRowLevelOperation... -> FlinkTableSink；DataStream API 路径：FlinkSink 抽象类派生 FixedBucketSink / DynamicBucketSink / AppendTableSink。

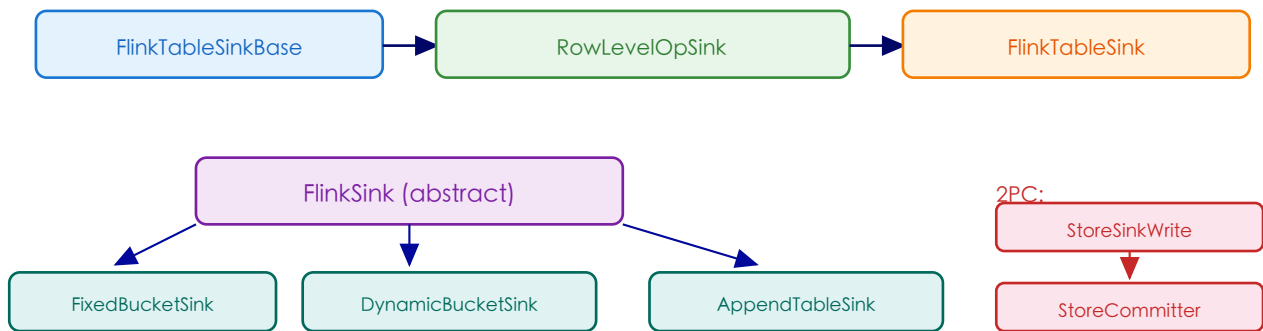


图3-2 Sink 类继承体系

3.4 Action CLI 框架

FlinkActions.main() -> ActionFactory.createAction() (SPI) -> Action.run() -> StreamExecutionEnvironment.execute()。框架注册 30+ 操作。

类别	Action	说明
表维护	CompactAction	触发Compaction
数据操作	MergeIntoAction	MERGE INTO语义
数据操作	DeleteAction	按条件删除
数据迁移	CloneAction	跨集群表克隆
标签管理	CreateTagAction	创建快照标签
CDC同步	MysqlSyncTableAction	MySQL单表同步
CDC同步	MysqlSyncDatabaseAction	MySQL整库同步

3.5 Procedure 框架

ProcedureBase 继承 Flink Procedure + Factory 接口，通过 SPI 发现。FlinkCatalog.getProcedure() 按名称查找并实例化。

```

CALL sys.compact('default.my_table');
CALL sys.create_tag('default.my_table', 'tag1', 1);
CALL sys.expire_snapshots('default.my_table', '2024-01-01');
  
```

3.6 Lookup Join 子系统

FileStoreLookupFunction (TableFunction) 在 open() 初始化本地 RocksDB 缓存，eval() 执行按主键查找。

- FullCacheLookupTable：全量缓存，适合小表维度
- PrimaryKeyLookupTable：按主键索引 + 增量更新
- NoPrimaryKeyLookupTable：无主键表查找（append-only）

- SecondaryIndexLookupTable : 二级索引 -> 主键 -> 数据

3.7 CDC 同步框架

paimon-flink-cdc 支持 MySQL/PostgreSQL/MongoDB/Kafka/Pulsar 五种数据源实时同步，采用六层架构。

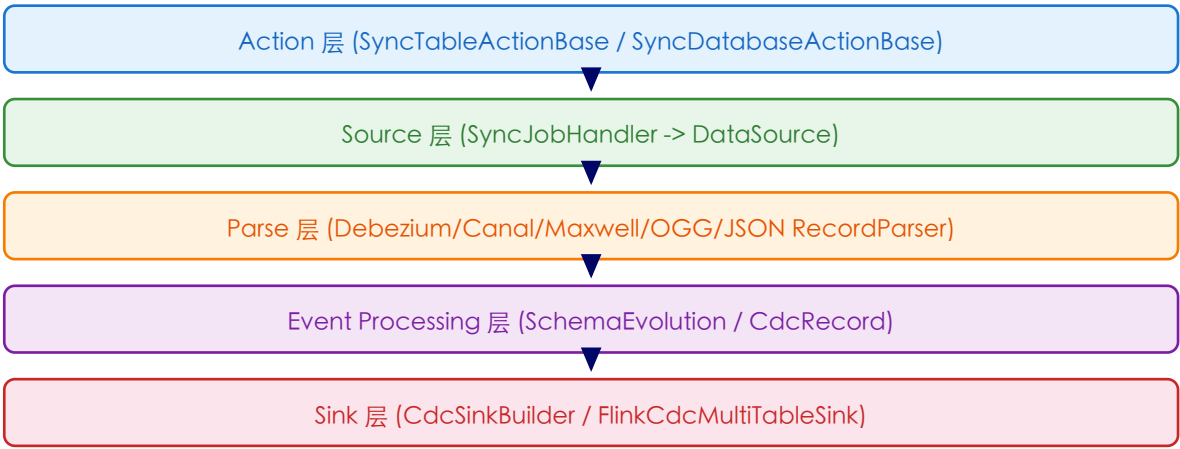


图3-3 CDC 六层架构图

3.7.1 两种数据库同步模式

- DIVIDED 模式：每表独立 Sink，表间完全隔离，适合表数量少
- COMBINED 模式：统一 FlinkCdcMultiTableSink，通过 tableId 路由，支持运行时动态建表

3.7.2 Schema 演进

- AddColumn：新增字段自动添加列
- UpdateColumnType：同族类型升级（INT->BIGINT, FLOAT->DOUBLE）
- UpdateColumnComment：同步列注释 canConvert()：类型兼容性检查

3.8 Flink 1.x/2.x 兼容层

组件	核心类	说明
类型增强	RowData/ArrayData接口	增加getVariant()方法
类型Stub	VariantType/VariantSerializer	Variant数据类型存根
运行时	RuntimeContextUtils	兼容工具
API适配	MultipleParameterToolAdapter	参数解析(下划线/连字符转换)

第4章 设计模式详解

4.1 抽象工厂模式 (Abstract Factory)

应用：FlinkCatalogFactory / FlinkTableFactory / AbstractFlinkTableFactory

FlinkCatalogFactory 创建 FlinkCatalog，FlinkCatalog 内部用 FlinkTableFactory 创建 Source/Sink。AbstractFlinkTableFactory 定义抽象方法，子类提供具体实现，形成完整工厂体系。

4.2 模板方法模式 (Template Method)

应用：FlinkSink / SynchronizationActionBase / FlinkTableSource

FlinkSink.sinkFrom() 定义 Writer->Committer 拓扑模板，子类 FixedBucketSink/DynamicBucketSink/AppendTableSink 实现特定步骤。SynchronizationActionBase.build() 定义 CDC 作业构建模板。

4.3 策略模式 (Strategy)

应用：StoreSinkWrite.Provider (按 ChangelogProducer 选写入策略) / BucketMode (选 Sink 实现) / LookupTable (四种查找策略)

4.4 Builder 模式

应用：FlinkSourceBuilder / FlinkSinkBuilder / CdcSinkBuilder / FlinkCdcSyncDatabaseSinkBuilder，封装复杂构建逻辑，链式调用，build() 产出对象。

4.5 SPI 服务发现模式

ActionFactory / ProcedureBase / CatalogFactory / RecordParser 均通过 META-INF/services 注册，ServiceLoader 运行时加载，添加新操作无需修改框架。

4.6 两阶段提交模式 (2PC)

应用：StoreSinkWrite / Committable / CommitterOperator / StoreCommitter

```
StoreSinkWrite.write(record) // ■■■■■■
StoreSinkWrite.prepareCommit(flush) // Phase1: Committable
CommitterOperator.notifyCheckpointComplete()
-> StoreCommitter.commit(committables) // Phase2: ■■■■
-> TableCommitImpl.commitMultiple() // ■ Snapshot
```

4.7 适配器模式 (Adapter)

FlinkRowData (Paimon InternalRow -> Flink RowData)、FlinkRowWrapper (反向)、Multiple ParameterToolAdapter (参数工具适配)，桥接内部数据格式与 Flink 运行时。

第5章 核心流程图

5.1 Source 数据读取流程

SELECT -> FlinkCatalog.getTable() -> FlinkTableFactory.createSource() -> PushDown -> Batch/Streaming Source -> SplitEnumerator + SourceReader -> RowData 流

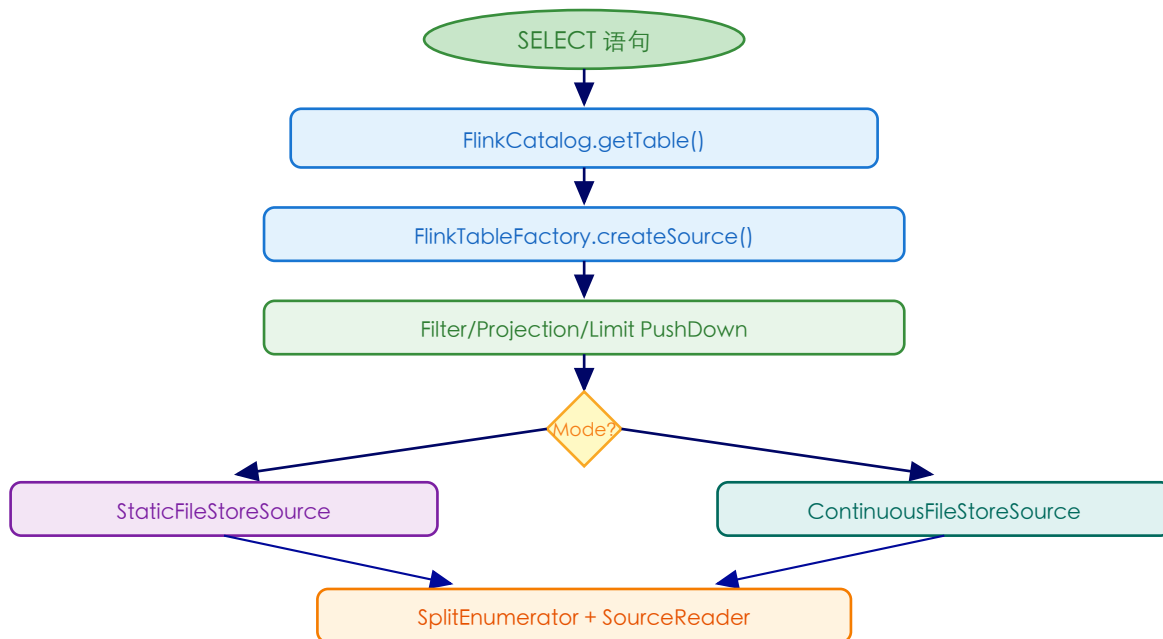


图5-1 Source 数据读取流程

5.2 Sink 数据写入流程

INSERT -> FlinkTableSink -> FlinkSinkBuilder -> BucketMode选择 -> TableWriteOperator -> prepareCommit -> CommitterOperator -> StoreCommitter -> Snapshot

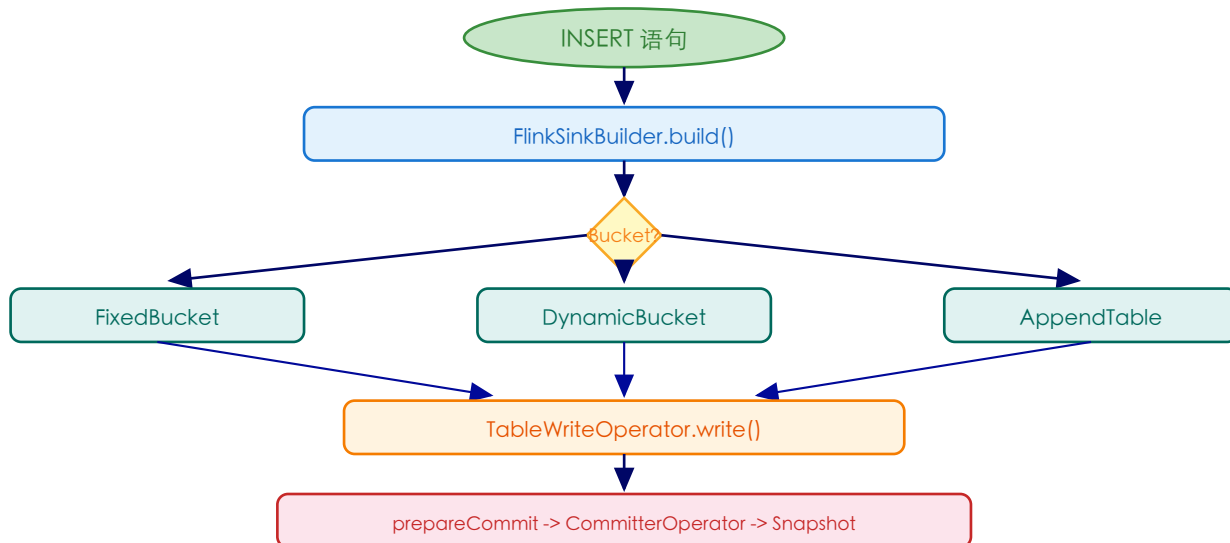


图5-2 Sink 数据写入流程

5.3 CDC 实时同步流程

CDC Action -> SyncJobHandler.provideSource() -> CDC Source -> RecordParser -> CdcRecord -> Schema演进检测 -> CdcRecordStoreWriteOperator -> StoreCommitter

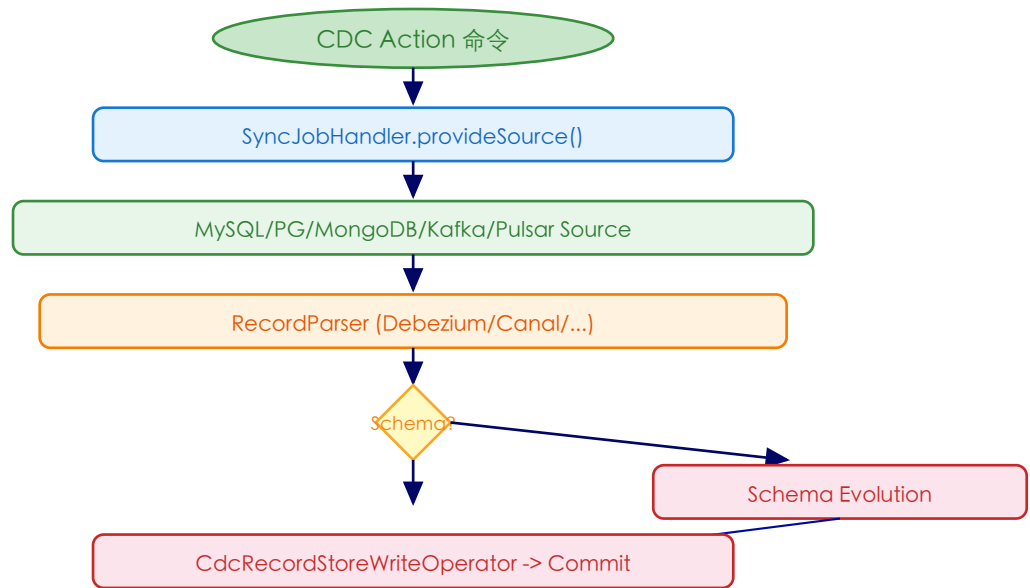


图5-3 CDC 同步流程

第6章 核心时序图

6.1 Source 创建与读取时序图

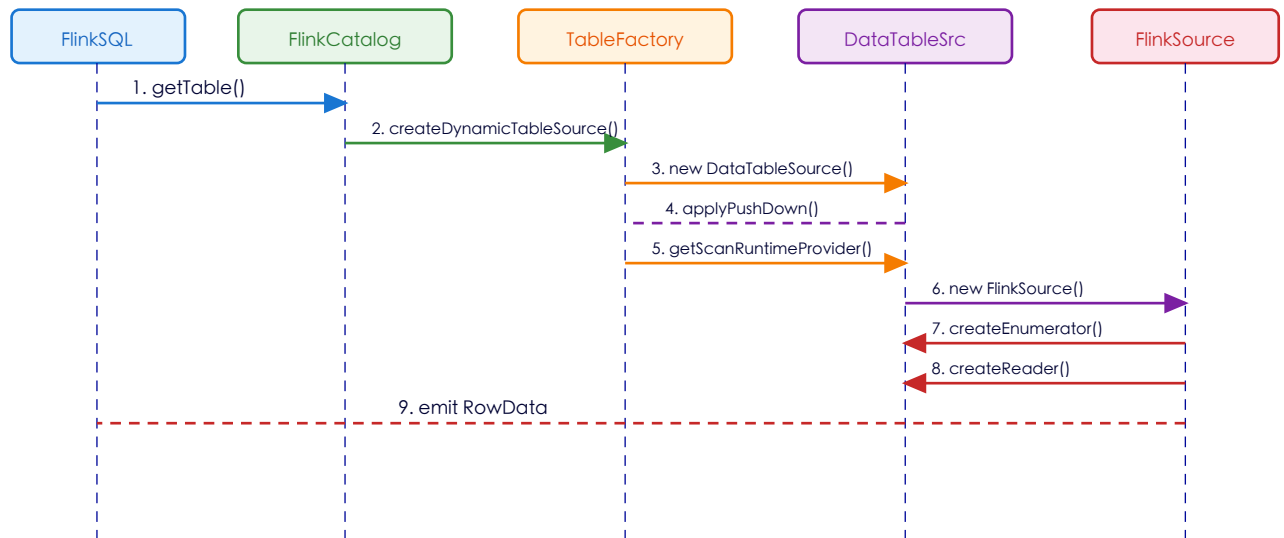


图6-1 Source 创建与读取时序图

6.2 Sink 两阶段提交时序图

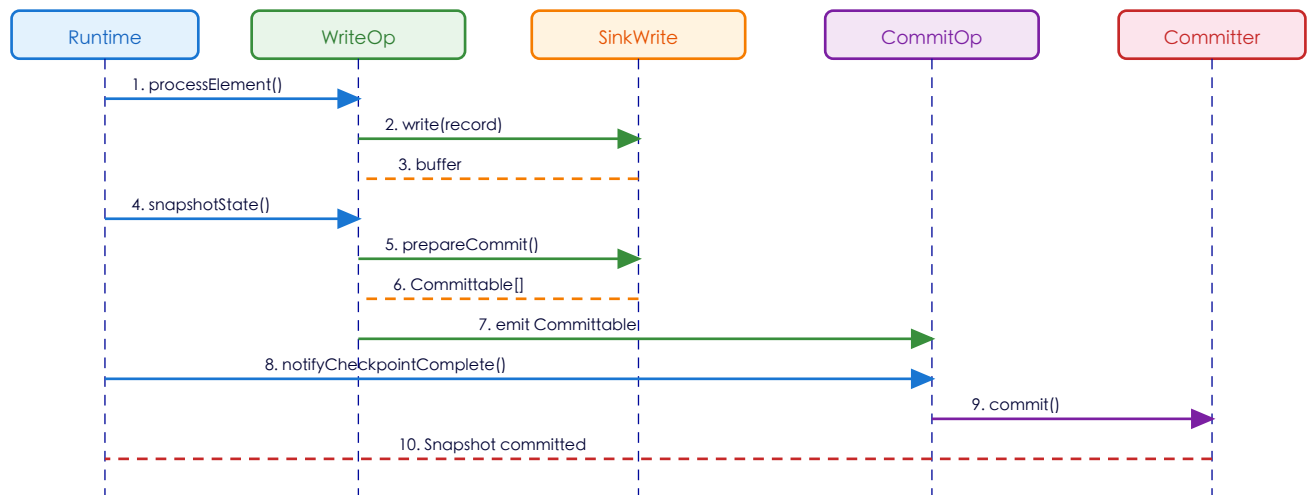


图6-2 Sink 两阶段提交时序图

第7章 关键场景调用链

7.1 场景一：Paimon 表创建

触发条件：执行 CREATE TABLE 语句

步骤	类.方法	说明
1	FlinkCatalog.createTable()	解析 DDL 选项，构建 Paimon Schema
2	FlinkCatalog.buildPaimonTable()	将 Flink Schema 转为 Paimon Schema
3	Catalog.createTable()	调用 Paimon Core 创建表元数据
4	SchemaManager.createTable()	写入 Schema 文件到文件系统

7.2 场景二：数据批量读取

步骤	类.方法	说明
1	FlinkTableFactory.createDynamicTableSource()	创建 DataTableSource
2	DataTableSource.applyFilters/applyProjection	下推谓词和投影
3	DataTableSource.getScanRuntimeProvider()	创建 FlinkSource
4	StaticFileStoreSource.createEnumerator()	扫描Snapshot获取Split列表
5	FileStoreSourceReader.pollNext()	按Split读取数据文件
6	FlinkRowData.wrap()	Paimon InternalRow 转 Flink RowData

7.3 场景三：数据流式写入

步骤	类.方法	说明
1	FlinkTableSink.getSinkRuntimeProvider()	创建 DataStreamSinkProvider
2	FlinkSinkBuilder.build()	根据 BucketMode 选择 Sink
3	FlinkSink.sinkFrom()	构建 Writer->Committer 拓扑

步骤	类.方法	说明
4	StoreSinkWriteImpl.write()	写入本地缓冲区(内存+溢写)
5	StoreSinkWriteImpl.prepareCommit()	Checkpoint时生成 Committable
6	CommitterOperator.notifyCheckpointComplete()	收集并触发提交
7	StoreCommitter.commit()	调用 TableCommitImpl 原子提交 Snapshot

7.4 场景四：MySQL CDC 单表同步

步骤	类.方法	说明
1	MysqlSyncTableAction.build()	构建 CDC 同步作业
2	SyncJobHandler.provideSource()	创建 MySQL CDC Source
3	DebeziumRecordParser.parse()	解析 Debezium 变更事件
4	CdcRecord.create()	封装为 CdcRecord(RowKind+data)
5	UpdatedDataFieldsProcessFunction	检测 Schema 变更
6	CdcRecordStoreWriteOperator.write()	写入 Paimon 表
7	StoreCommitter.commit()	提交 Snapshot

7.5 场景五：Compaction 触发

步骤	类.方法	说明
1	CompactAction.build() / CompactProcedure.call()	触发入口
2	FlinkSourceBuilder.buildSource()	创建 Compaction Source
3	CompactorSourceBuilder.build()	构建 Compaction Split
4	FlinkSink.sinkFrom()	创建 Compaction Writer
5	CompactManager.submitCompaction()	提交 Compaction 任务
6	MergeTreeCompactTask.call()	执行文件合并/排序

第8章 设计亮点与总结

8.1 设计亮点

- 批流一体：同一套 Source/Sink 体系统一支持 Batch 和 Streaming 两种模式，通过 StaticFileStoreSource/ContinuousFileStoreSource 自动切换
- SPI 驱动的可扩展性：Action、Procedure、RecordParser 等全部采用 SPI 注册，新增功能无需修改框架代码
- 两阶段提交保证 Exactly-Once：StoreSinkWrite -> Committable -> CommitterOperator -> StoreCommitter 的完整 2PC 链路
- 四种 PushDown 优化：Filter/Projection/Limit/Aggregate PushDown 层层递进，最大化减少数据传输
- CDC Schema 自动演进：源表结构变更自动同步到 Paimon 表，支持类型升级兼容性检查
- 多版本 Flink 兼容：通过 paimon-flink1-common / paimon-flink2-common 兼容层支持 Flink 1.x 和 2.x
- 多种 Bucket 策略：Fixed/Dynamic/Unaware 三种模式适应不同数据分布场景

8.2 设计限制

- CDC 整库同步的 COMBINED 模式复杂度：多表共享 Sink 链路，故障隔离性较弱
- Lookup Join 全缓存模式内存压力：大表维度可能导致 RocksDB 占用过多本地磁盘
- Schema 演进仅支持同族类型升级：不支持跨族类型变更（如 STRING->INT）

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	四层架构清晰，职责分明，460+ 源文件组织有序
设计模式运用	★★★★★	Factory/Template/Strategy/Builder/SPI/2PC/Adapter 全面运用
可扩展性	★★★★★	SPI 驱动，新增 Action/Procedure/Parser 零修改框架
批流一体性	★★★★★	统一 Source/Sink 体系无缝切换 Batch/Streaming
CDC 集成深度	★★★★☆	五种数据源 + 七种消息格式 + Schema 演进，但 COMBINED 模式稍复杂
文档完整性	★★★☆☆	代码注释较少，部分复杂逻辑缺乏设计说明

8.4 总结

paimon-flink-1.20 模块通过三个核心子模块（paimon-flink-common 460+ 文件、paimon-flink1-common 16 文件、paimon-flink-cdc 170+ 文件）构建了功能完整的 Apache Flink 集成层。其设计体现了高度的工程素养：四层分层架构保证了关注点分离；SPI 驱动的

扩展机制提供了优秀的可插拔性；两阶段提交保证了数据一致性；批流一体设计简化了用户心智模型。

- CDC 同步框架支持五种主流数据源和七种消息格式，覆盖了绝大多数实时数据同步场景。整体而言，这是一个架构设计优秀、功能覆盖全面的 Flink 连接器实现。